

General

Architecture Pattern: describes software organization, its subsystems, each subsystem responsibilities, rules, & guidelines

Design Pattern: refine/support the relationships among software subsystems or components

Behavioral Pattern: dynamic interactions among families of classes, distribute responsibility

→ Class: using inheritance

→ Object: using delegation

Structural Pattern: Decouple interface & class implementation, compositions

→ Class: using inheritance

→ Object: using delegation

Nico

Bokhari

CIS476

Final

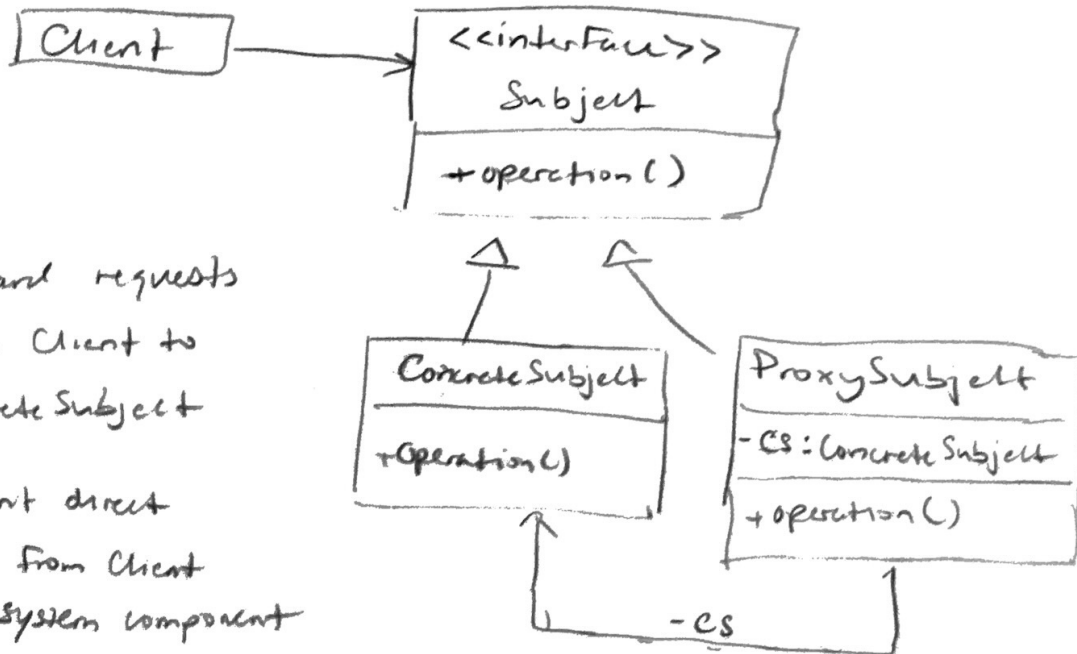
Help Sheets

Pg 1

Proxy - Object Structural

* Most commonly used Design Pattern

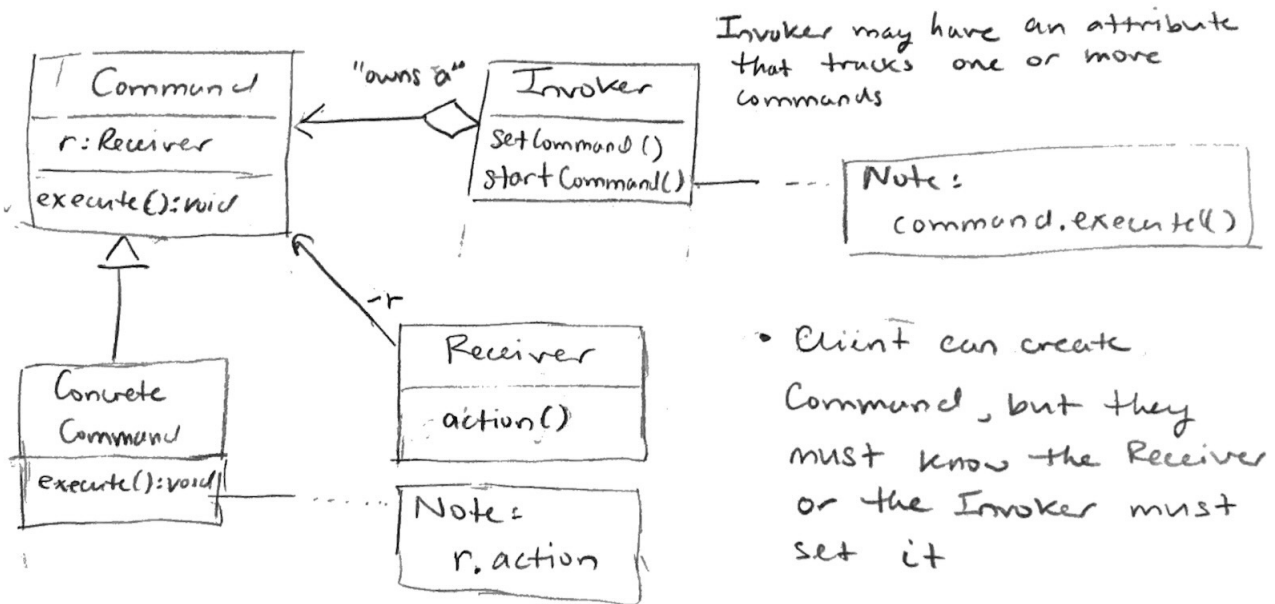
- Intermediary between clients and target system object



- Forward requests from Client to Concrete Subject
- Prevent direct link from Client to system component

Command - Object Behavioral

- Software allows client requests to be stored, queued, and processed
 - Client requests are "Command" objects with particular receiver
 - Invoker queues Commands, and executes them in order
 - Receiver performs actual operation
- ★ Decouple operation invoker from actual operation itself

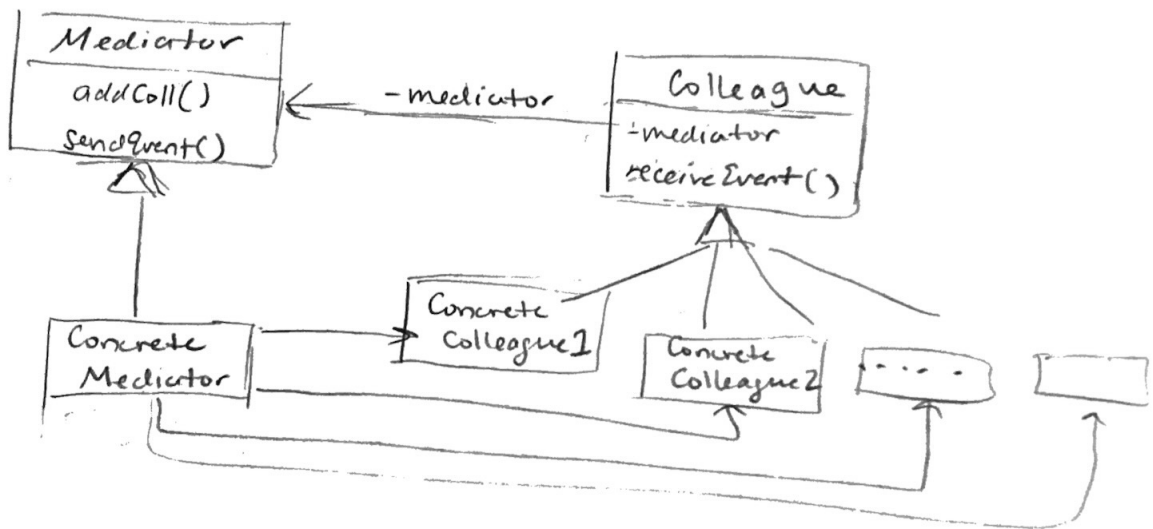


- Client can create Command, but they must know the Receiver or the Invoker must set it

Mediator - Object Behavioral

Pg 3

- Many objects in a system communicate
- Avoid deep coupling with each object keeping individual references
- Objects use mediator to communicate
- Mediator has most knowledge on all events and interacting components
- ★ Abstracts away object cooperation

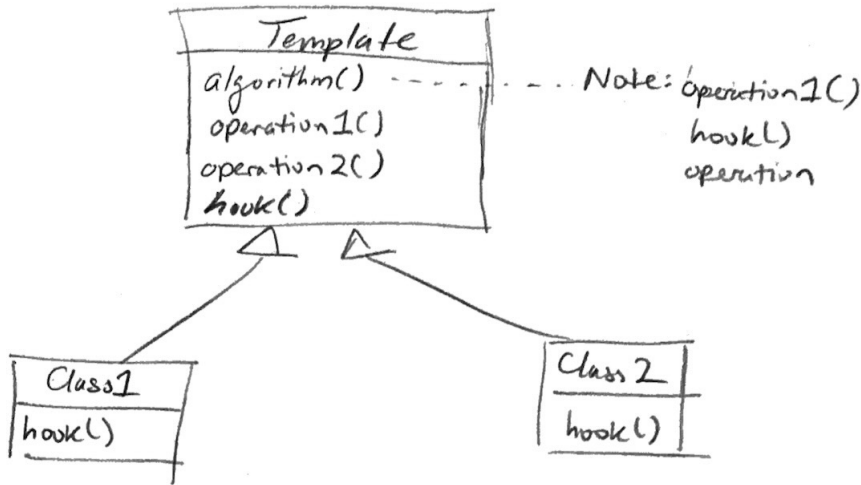


- Concrete Mediator keeps a list/structure that stores all colleagues
- if one colleague sends event, it is sent using the mediator reference in Colleague class, calling `sendEvent()`
- All colleagues (not sender) notified

Template - Class Behavioral

Pg 4

- Subclasses can share template method, but implement their own detailed steps

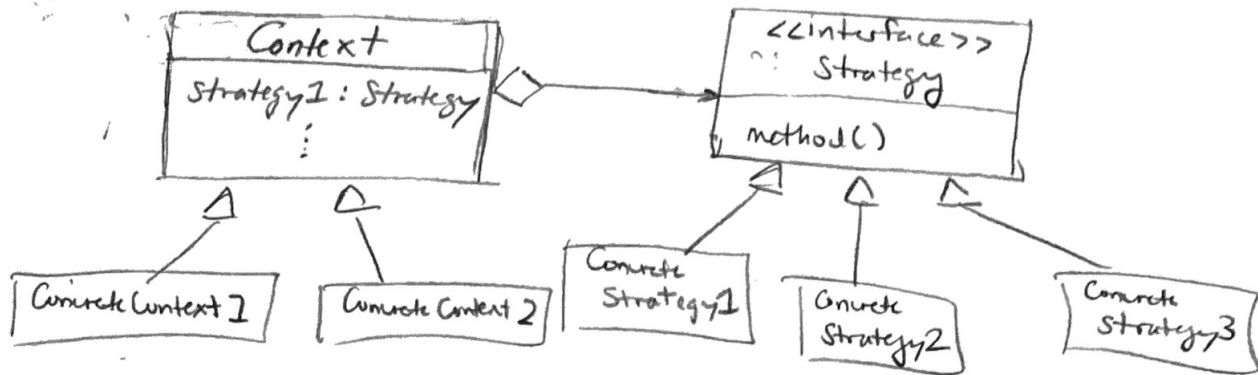


- If a software component follows a similar set of steps, with some details tied to the component's implementation, Template used so all components share algorithm structure.
- Avoid code duplication $\longleftrightarrow$ Code reuse
- hook methods used to specify points where extension is allowed

Strategy - Object Behavioral

Pg 5

- Encapsulate behaviors into interfaces
- Objects delegate behavior to subclasses of interface

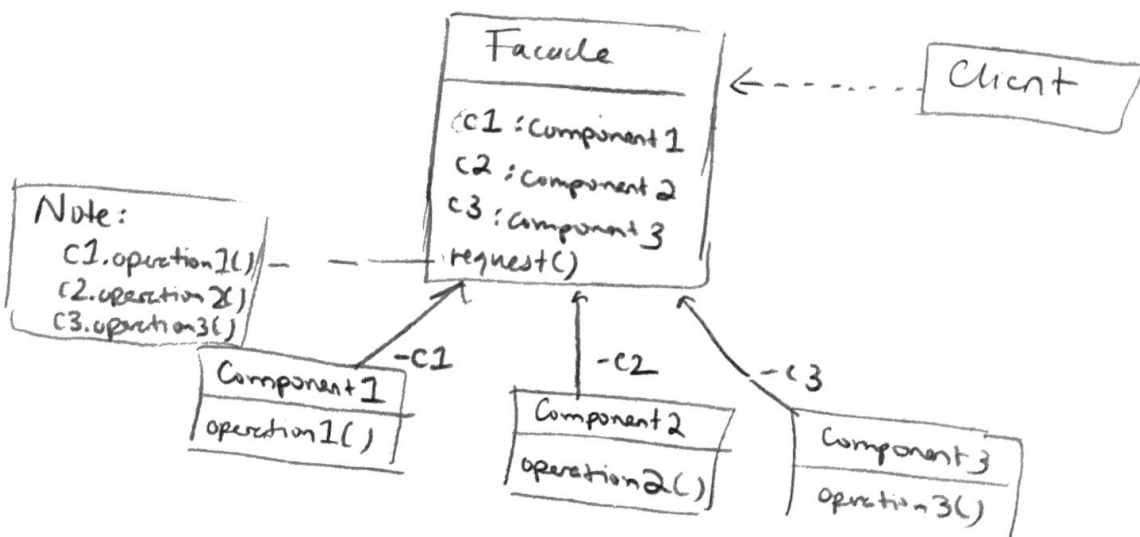


- Abstracts away complex algorithm specific details that differs given a **ConcreteContext**.
- Alternative to subclassing **Context** into variety of behaviors
 - ↳ Increase in # of objects used for tracking behavior

Facade - Object Structural

pg 6

- Abstract away complexity of interacting with system.
- Provides easier interface to use system
- Facade acts as intermediary between Client & system
- Facade delegates Client requests to actual system components
- System has no knowledge of Facade (no references)



Facade vs. Mediator vs. Command

- Client interfaces system thru Facade
- Abstracts away system details

- Abstracts away component cooperation

- Client interacts with system using an Invoker and Commands
- For queuing Client requests

Structural

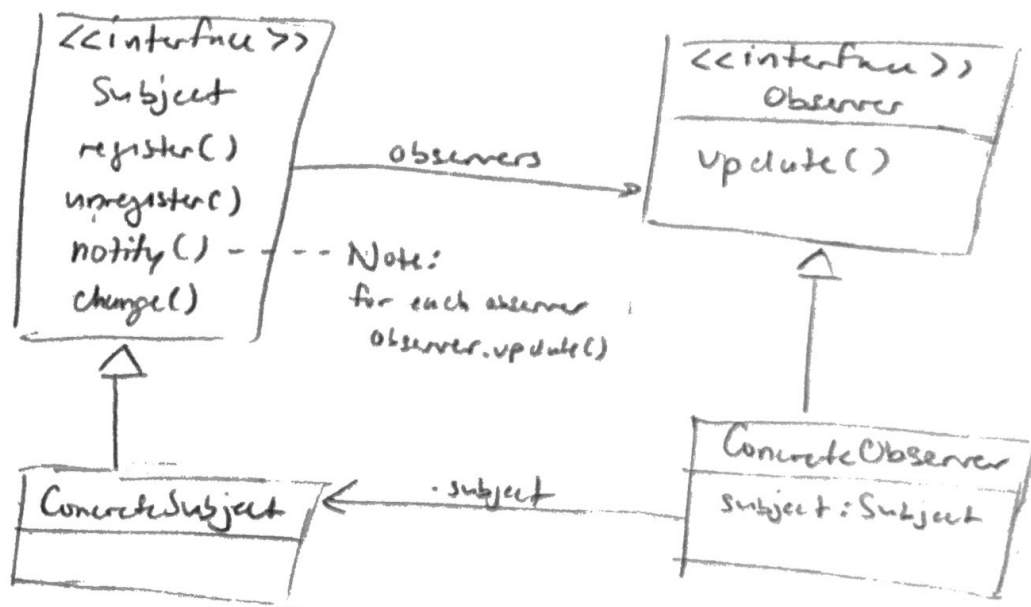
Behavioral

Behavioral

Observer - Object Behavioral

Pg 7

- Prevent deep coupling from one-to-many dependency
- Many objects automatically updated from one object's state change.
- Subject only aware that it has observers to notify given a state change
- Observers keep reference to Subject, register themselves to get updates
- Subject keeps list of observers, may update all or conditionally update specific observers



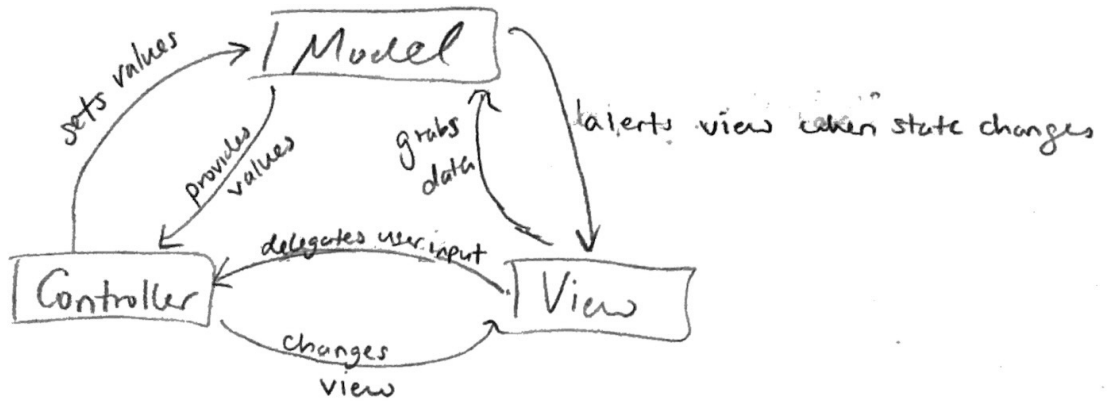
Push mode

→ send data directly to observer

Pull mode

→ Observer notified of data change, must pull data from other source

Architecture Pattern for Interactive Systems



Many classes may be used in software for many views, models, and controllers

